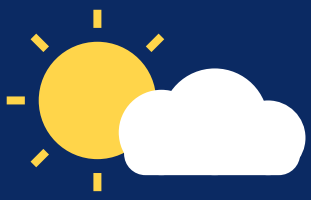


DATA ENGINEERING PROJECT REPORT

Open-Meteo Weather API

End-to-End ETL Pipeline

Data quality checks, SQL scripts, and pipeline and field-mapping documentation
Hourly weather for Hyderabad, from API to SQL Server, built with Python

1 Extract ▶ 2 Transform ▶ 3 Validate ▶ 4 Export ▶ 5 Load

Report date: 8 November 2026

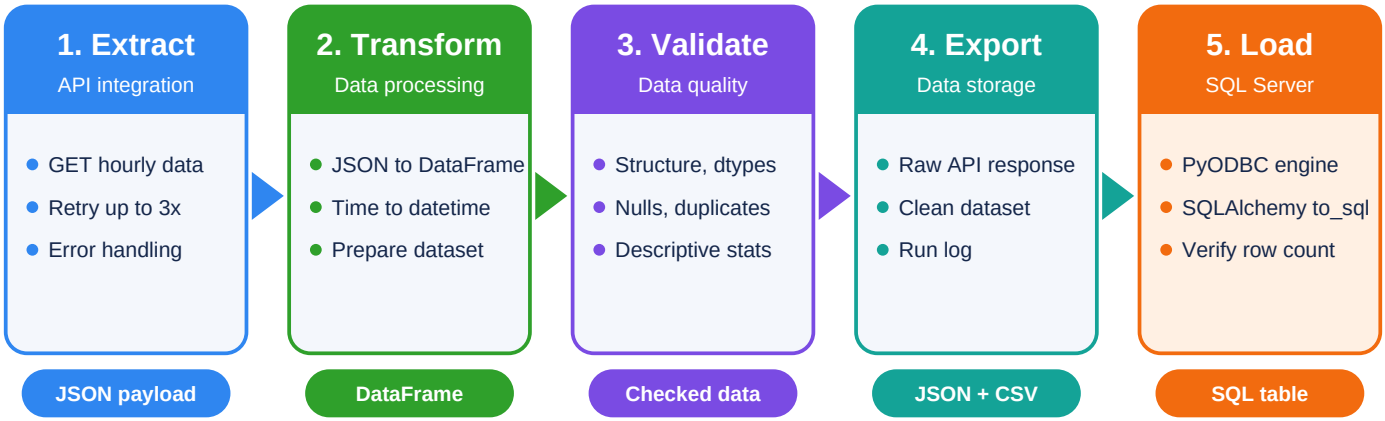
24	5	0	0
weather records	columns	missing values	duplicate records

Source	Open-Meteo Forecast API (hourly variables)
Location	Hyderabad (17.3850 N, 78.4867 E), timezone Asia/Kolkata
Destination	SQL Server database WeatherData, table dbo.WeatherHourlyData
Run result	**SUCCESS**, 24 of 24 rows loaded and verified
Tools	Python, Requests, Pandas, PyODBC, SQLAlchemy, Jupyter Notebook

INSIDE: 1 Pipeline overview | 2 Source and configuration | 3 Field mapping | 4 Data quality checks | 5 SQL scripts | 6 Findings and next steps

1 Pipeline Overview

The notebook runs five stages in order. Each stage writes to a log file, so a failed run can be traced to the stage that broke. The final stage reads the table back to confirm the load.



Stage summary

Stage	Notebook steps	What happens	Output
Extract	4, 5, 6, 7	Builds the request (location, hourly variables, timezone), mounts a retry session, calls the API with a 30 s timeout and checks the HTTP status.	JSON payload
Transform	8, 11	Converts the hourly JSON block into a DataFrame and converts the time text into real datetime values.	DataFrame (24 x 5)
Validate	9, 10	Checks shape, dtypes, missing values, duplicate rows and descriptive statistics. Logs the result.	Quality report
Export	12	Writes the raw API response and the clean dataset to files.	api_response.json, hyderabad_weather_data.csv
Load	15 to 20	Connects with PyODBC, builds a SQLAlchemy engine, loads with to_sql, then counts rows in SQL Server.	dbo.WeatherHourlyData
Monitor	3, 13, 21	Logging to file, execution time, records processed and a final summary.	weather_pipeline.log

Reliability built in: automatic retries on HTTP 429, 500, 502, 503 and 504 (3 attempts, growing wait), a 30 second timeout, *raise_for_status* to catch bad responses, and *logger.exception* to record the full error before the run stops.

2 Source and Configuration

API request

Setting	Value	Why
Endpoint	https://api.open-meteo.com/v1/forecast	Free Open-Meteo forecast service
Latitude / Longitude	17.3850 / 78.4867	Hyderabad. The API snaps to its nearest grid point (17.3989, 78.4571, elevation 505 m)
Hourly variables	temperature_2m, relative_humidity_2m, wind_speed_10m, precipitation	The four measures kept for analysis
forecast_days	1	One full day, 24 hourly rows
timezone	Asia/Kolkata (UTC+05:30)	Timestamps in local time
Retry policy	total=3, backoff_factor=1, GET only	Handles temporary server errors
Timeout	30 seconds	Stops a hung request

Output files

File	Content	Created by
api_response.json	Full raw API response, indented	json.dump (step 12)
hyderabad_weather_data.csv	Clean 24 x 5 dataset, no index	df.to_csv (step 12)
weather_pipeline.log	Timestamped INFO and ERROR messages	logging (step 3)

SQL Server connection

Setting	Value
Driver	ODBC Driver 17 for SQL Server (found with pyodbc.drivers())
Authentication	Windows authentication (Trusted_Connection). No passwords stored in the notebook.
Database / Table	WeatherData / dbo.WeatherHourlyData
Load method	pandas to_sql through a SQLAlchemy mssql+pyodbc engine, if_exists = append, index = False

3 Field Mapping

How each value travels from the API response to the database. Read each row left to right.

API field (hourly)	Unit	DataFrame column	Pandas dtype	SQL column	SQL type
time	iso8601 text	time	object, then datetime64[ns]	time	DATETIME2(0)
temperature_2m	degrees C	temperature_2m	float64	temperature_2m	FLOAT
relative_humidity_2m	%	relative_humidity_2m	int64	relative_humidity_2m	INT
wind_speed_10m	km/h	wind_speed_10m	float64	wind_speed_10m	FLOAT
precipitation	mm	precipitation	float64	precipitation	FLOAT

Transformation rules

#	Rule	Code
1	Take only the hourly block of the response; the other keys (latitude, units, timezone) stay in the JSON file	pd.DataFrame(data['hourly'])
2	Parse time text such as 2026-11-08T00:00 into a datetime	pd.to_datetime(df['time'])
3	Column names are kept exactly as the API names them, so mapping is one to one	no rename step
4	Row order follows the API (hour 00:00 to 23:00), no sorting or filtering	none

Data lineage

Open-Meteo
hourly JSON

DataFrame
24 x 5, typed

CSV + JSON
files on disk

dbo.WeatherHourlyData
24 rows

4 Data Quality Checks

Every check below was run in the notebook. Results are copied from the real output of the run.

#	Check	Method	Result	Status
1	Structure	df.shape, df.info(), df.columns	24 rows, 5 columns, expected names present	PASS
2	Data types	df.info(), df.dtypes	time is datetime64[ns]; others float64 or int64	PASS
3	Missing values	df.isnull().sum()	0 in every column	PASS
4	Duplicate records	df.duplicated().sum()	0	PASS
5	Completeness	Row count against 24 hours	24 of 24 hourly rows (00:00 to 23:00)	PASS
6	Value ranges	df.describe()	All values plausible for Hyderabad (table below)	PASS
7	API health	raise_for_status, retries, log	Request succeeded; 'Data extracted successfully' logged	PASS
8	Load reconciliation	SELECT COUNT(*) in SQL Server	24 rows in table = 24 rows processed	PASS
9	Precipitation	df.describe()	Every value is 0.0 mm (a dry day). Expected, but worth watching	NOTE

Statistical profile (df.describe())

Statistic	temperature_2m (C)	relative_humidity_2m (%)	wind_speed_10m (km/h)	precipitation (mm)
count	24	24	24	24
mean	26.06	40.29	8.78	0.00
std	3.37	9.03	1.95	0.00
min	21.60	29.00	6.00	0.00
25%	22.95	31.75	7.30	0.00
50%	25.40	40.00	8.35	0.00
75%	29.08	51.00	9.80	0.00
max	31.10	52.00	13.60	0.00

Reading the numbers: temperature moved from 21.6 C (06:00, coolest) to 31.1 C (14:00, warmest). Humidity was lowest in the afternoon (29%) and highest before sunrise (52%). Wind peaked at 13.6 km/h around 16:00. These patterns are what a clear November day in Hyderabad should look like.

5 SQL Scripts

Ready-to-run T-SQL for the WeatherData database. Script 1 is a recommended table definition that matches the field mapping and blocks duplicate hours. Run them in SQL Server Management Studio; if the table already exists from the notebook, Script 1 skips creation.

Script 1: create the table

```
USE WeatherData;
GO
IF OBJECT_ID('dbo.WeatherHourlyData', 'U') IS NULL
CREATE TABLE dbo.WeatherHourlyData (
    [time]               DATETIME2(0) NOT NULL,   -- local time, Asia/Kolkata
    temperature_2m       FLOAT        NULL,       -- degrees C
    relative_humidity_2m INT          NULL,       -- percent
    wind_speed_10m       FLOAT        NULL,       -- km/h
    precipitation        FLOAT        NULL,       -- mm
    CONSTRAINT PK_WeatherHourlyData PRIMARY KEY ([time])
);
GO
```

Script 2: verify the load (matches notebook step 20)

```
SELECT COUNT(*)    AS total_rows,
       MIN([time]) AS first_hour,
       MAX([time]) AS last_hour
FROM dbo.WeatherHourlyData;

SELECT TOP (5) * FROM dbo.WeatherHourlyData ORDER BY [time];
```

Script 3: data quality checks in SQL

```
-- 3a. Missing values (expect 0 in every column)
SELECT SUM(CASE WHEN [time]               IS NULL THEN 1 ELSE 0 END) AS null_time,
       SUM(CASE WHEN temperature_2m       IS NULL THEN 1 ELSE 0 END) AS null_temp,
       SUM(CASE WHEN relative_humidity_2m IS NULL THEN 1 ELSE 0 END) AS null_humidity,
       SUM(CASE WHEN wind_speed_10m       IS NULL THEN 1 ELSE 0 END) AS null_wind,
       SUM(CASE WHEN precipitation        IS NULL THEN 1 ELSE 0 END) AS null_precip
FROM dbo.WeatherHourlyData;

-- 3b. Duplicate hours (expect no rows)
SELECT [time], COUNT(*) AS copies
FROM dbo.WeatherHourlyData
GROUP BY [time]
HAVING COUNT(*) > 1;

-- 3c. Out-of-range values (expect no rows)
SELECT *
FROM dbo.WeatherHourlyData
WHERE relative_humidity_2m NOT BETWEEN 0 AND 100
   OR temperature_2m       NOT BETWEEN -60 AND 60
   OR wind_speed_10m < 0
   OR precipitation  < 0;

-- 3d. Completeness: 24 hourly rows per day
SELECT CAST([time] AS DATE) AS weather_date,
       COUNT(*)             AS hours_loaded,
       CASE WHEN COUNT(*) = 24 THEN 'PASS' ELSE 'CHECK' END AS status
FROM dbo.WeatherHourlyData
GROUP BY CAST([time] AS DATE);
```

Script 4: daily summary view for reporting

```
GO
CREATE OR ALTER VIEW dbo.vw_WeatherDailySummary AS
SELECT CAST([time] AS DATE)              AS weather_date,
       MIN(temperature_2m)               AS min_temp_c,
       MAX(temperature_2m)               AS max_temp_c,
       ROUND(AVG(temperature_2m), 1)     AS avg_temp_c,
       AVG(relative_humidity_2m)         AS avg_humidity_pct,
       MAX(wind_speed_10m)               AS max_wind_kmh,
       SUM(precipitation)                AS total_precip_mm
FROM dbo.WeatherHourlyData
GROUP BY CAST([time] AS DATE);
GO
```

6 Findings and Next Steps

Reviewing the run output, these points would make the pipeline stronger. None of them stopped the run.

#	Finding	Why it matters	Suggested fix
1	Execution time shows 3817.12 s	The timer starts in the logging cell and stops in a later cell, so it also counts the minutes between running cells. The pipeline itself takes seconds.	Time the stages inside one cell or one function, using time.perf_counter()
2	Step 14 preview shows 2026-11-07 rows	The API response printed in step 7 covers 2026-11-08, so the preview is output from an earlier run.	Re-run step 14 after a full run so the report matches the data
3	Load mode is append	Running the notebook twice would add the same 24 hours again (48 rows).	Add the primary key from Script 1, or load only hours not already in the table
4	Precipitation is all zero	Correct for a dry day, but an all-zero column can also mean a missing feed.	Add a check that flags a long run of zeros over several days
5	Duplicate step numbers	The notebook has two step 1 and two step 17 headings.	Renumber the markdown headings

Roadmap

Automate
Schedule the run daily with Windows Task Scheduler or Airflow.

Scale
Add a city column and loop over several locations.

Guard
Fail the run automatically when a quality check does not pass.

Report
Connect Power BI to the daily summary view.

Goal achieved: a reliable, automated and scalable path from a public weather API to a SQL Server table, with logging, retries, quality checks and a verified row count at the end.